

Project #5, SuperMarket App

Objectives

To practice Node.js, setting routes using express and JSON data processing.

Overview

For this project you will implement an app called Supermarket that allow users to buy food online. You will define a server (using Node.js) that will allow users to see the items for sale and to place an order. Information about items available will be provided in JSON files that the server will access. A video showing some of the system functionality can be found at Application Video. You should watch this video before you continue reading. Please read the submission instructions below (they are not the usual ones).

Announcements

In previous semesters we had students using versions of this project posted online. I want to warn you that I am aware of these sources. Please abide by the academic integrity rules defined in the class syllabus. Thank you.

Grading

- (9%) One ES6 class (like in Java) defined and used. The class has least one private instance variable, a constructor and at least one method (besides the constructor). What you used the class for, it is up to you.
- (10%) Server command line interpreter
 - (2%) Stopping server with "stop".
 - (4%) Displaying items list when "itemsList" is entered.
 - (2%) "Invalid command: " followed by command is displayed for a command other than "stop" or "itemsList".
 - (2%) Displaying the message "Usage supermarketServer.js jsonFile" when an incorrect number of command line arguments are provided.
- (16%) Provided template files are used to generate the application pages.
- (5%) Not using iteration statements (i.e., for(i = 0; i <), while, or do-while) while processing an array. You need to use array functions similar to forEach, reduce, find, etc.
- (40%) Placing Order Functionality
 - (10%) Correct item's list is displayed after information is provided (using one of the JSON files provided).
 - (10%) Correct total cost generated (using one of the JSON files provided).
 - (10%) Correct item's list is displayed after information is provided (using our own JSON file).
 - (10%) Correct total cost generated (using our own JSON file).
- (20%) Catalog Functionality
 - (10%) Correct item's list is displayed after information is provided (using one of the JSON files provided).

- (10%) Correct item's list is displayed after information is provided (using our own JSON file).

Code Distribution

The project's code distribution is available in your upstream. When you unzip the file, you will find a folder called Supermarket. This folder includes:

- templates folder - Template files for the application. Do not modify these files (except to update the port number).
- itemsList.json - Items JSON file.
- itemsListTwo.json - Another items JSON file.
- supermarketServer.js - Empty file where you will implement your server.
- Support files for node (e.g., package.json)

Specifications/Requirements

Module Installations

Before you start the project, you need to run `npm init` in the project folder (Supermarket). Use the default values that are provided. Running `init` will create the `package.json` file.

The modules you need for this project are `express` (`npm i express`), `ejs` (`npm i ej`s) and `body-parser`. Please use `nodemon`, as you develop your code (it will simplify the development process). Remember that `nodemon` restarts the server after any server code change. Unlike `express`, `ejs`, and `body-parser`, install `nodemon` as a development dependency (`npm i nodemon --save-dev`). The only modules you can use are `express`, `ejs`, `body-parser`, `nodemon` and any other module discussed in lecture.

Items Information

Information about the items a supermarket sells will be provided via a JSON file (e.g., `itemsList.json`). The `"itemsList"` property represents an array of the items the supermarket sells. The `"itemsListName"` property is the name of the list (you can ignore it as it will not be used in the project).

Command Line Interpreter

You should start developing your application by implementing the command line interpreter that is part of the `supermarketServer.js` file. Users will start the server by executing `node supermarketServer.js` followed by a file with JSON information about items sold by the supermarket. For example, to start the server with the data available in `itemsList.json` the user will type on the command line: `node supermarketServer.js itemsList.json`

About the command line interpreter:

- The command line interpreter will use the prompt "Type itemsList or stop to shutdown the server: ".
- The server will stop when "stop" is entered. The message "Shutting down the server" should be displayed when the server is stopped. Use `process.exit(0)` to stop the server.
- The items in the JSON file loaded are displayed when "itemsList" is entered.
- "Invalid command: " followed by the command entered, is displayed for a command other than "stop" or "itemsList".
- Displays the message "Usage supermarketServer.js jsonFile" when an incorrect number of command line arguments are provided.

Defining and Processing Endpoints (path and specific HTTP method)

Using express, you will define endpoints/routes/URLs that will generate the expected processing. Assuming `app` is the express request handler function, the endpoints you need to implement are:

- `app.get("/", (request, response) => { /* You implement */ });` - This endpoint renders the main page of the application and it will display the contents of the `index.ejs` template file.
- `app.get("/catalog", (request, response) => { /* You implement */ });` - This endpoint displays the `displayItems.ejs` template with the table of items available.
- `app.get("/order", (request, response) => { /* You implement */ });` - This endpoint displays the `placeOrder.ejs` template with the table of items available.
- `app.post("/order", (request, response) => { /* You implement */ });` - This endpoint will process the submission of the `placeOrder` form, retrieving the order values and processing the order. Processing an order requires displaying the `orderConfirmation.ejs` template with a table that includes the items to be purchased, along with their cost. The last table row has the sum of all the items in the order.

Template files

We have provided the template files you need to use while processing endpoints. In those files, except for `index.ejs`, you will find variable(s) (e.g., `<%- orderTable %>`) that will be replaced with the appropriate contents. Class examples illustrate how to use the `render` method (e.g., `response.render()`) with an `ejs` file and an object that has values that will replace the `ejs` file variables. Do not modify the provided template files (except to update the port number).

Format for Output Messages and Tables

Take a look at the [application video here](#) to identify the format expected for output messages and tables.

Data Validity

You can assume the provided JSON files are correct. Also, you can assume users will provide the expected data (the application does not have to generate any error messages for invalid data provided by the user).

Port Number

Your server will use 5000 as the port number. The application main page will be accessed via the endpoint `http://localhost:5000/`. If your computer has a conflict with port number 5000 (Macs seem to have this problem), then select a different port number (e.g., 5001) and change the port number in the `templates/placeOrder.ejs` file. Also, make sure the message "Web server started and running at `http://localhost:...`" has the correct port number. If you change the port number, you don't have to change it back once you are ready to submit your work.

Miscellaneous

- Check the Piazza clarifications for this project often.
- Do not use `fetch` for this project. Use the `fs` module `readFileSync()` method to read the `items list` file and then `JSON.parse()` to generate the required object.
- If the Node.js version you have allows you to run class examples, you should be fine, although we are concerned that during grading, we may run into problems. Therefore, try to use the most recent version of Node.js/npm. Your npm update may fail unless you have an appropriate Node.js version; update Node.js first, and then npm.
- Feel free to use any code presented in lecture.
- You can use the method `toFixed()` (e.g., `cost.toFixed(2)`) to generate a currency value with two decimal places.
- There is no need to use `express.Router` in this project.
- To run nodemon in powershell (PC) you need to change execution permissions by:
 - Opening "Windows Powershell (Admin)"
 - `Set-ExecutionPolicy Unrestricted`
 - Warning: previous step allows any script to execute
- Truncate the sum using `toFixed()` after you have added the costs.
- Do not worry if the user doesn't select any items in their order (you can assume they will always make a selection).
- Make sure you have the most recent versions of Node.js and npm. Some students reported problems defining class while using old versions of Node.js/npm
- We do not use XAMPP for this project. You download the Supermarket folder anywhere in your computer.
- Add "use strict"; to the top the .js file as we want to use strict mode.

Submission:

- To submit your project:
 - Remove the `node_modules` folder. This folder is large and we can recreate it while grading (by using `npm i`).
 - Push to upstream as normal!